Supplementary Material for: Honeyimage Generation-Based Graphical Authentication: Deep Hybrid Learning with Visual Flatness Optimization

Zena M. Saadi, Ahmed T. Sadiq**, Omar Z. Akif ****

** Computer Sciences Collage, University of Technology, Iraq .*
*** Computer Sciences Collage, University of Technology, Iraq .*
**** Department of Computer Science, College of Education for Pure Science -Ibn al-Haitham, University of Baghdad, Iraq.*
Emails: 1 cs.22.15@grad.uotechnology.edu 2 ahmed.t.sadiq@uotechnology.edu.iq
3 omar.z.a@ihcoedu.uobaghdad.edu.iq

S1. Additional Experimental Results (earlier trials)

This subsection presents the architectural and training parameters of the CNN model after conducting a series of preliminary experimental trials. Multiple CNN configurations were designed, trained, and evaluated according to predefined performance criteria, and the obtained results were systematically recorded and stabilized in the subsequent tables. Based on a comparative analysis of these experiments, the model that achieved the most favorable performance was selected. Accordingly, this optimized CNN model was adopted for Honeyimage generation and utilized as the final model.

Table A.1. CNN Approach1 Model Parameters

Layer/Type	Parameter	Value	Description
Conv2D	kernel	(3×3×64), (3×3×128), (3×3×256), (3×3×512)	This filter is used to extract features from images.
	bias	(64), (128), (256), (512)	A per-unit offset that helps the neuron's output, allowing networks to fit the data more flexibly.
	activation	relu (Rectified Linear Unit)	Suppresses negative values, retaining positive ones (enabling deep networks to learn efficiently).
	dilation_ rate	(1,1)	Controls the spacing between kernel elements, (1,1) means no dilation (standard)
	filters	64, 128, 256, 512	Number of kernels (feature detectors) in a convolutional layer; determines how many features are extracted.
	groups	1	Number of groups for grouped convolution; 1 means standard convolution
	padding	same	"same" keeps output size the same as input by adding zero-padding as needed
	kernel_ size	(3,3)	Height and width of the convolution window.
	strides	(1,1)	How many pixels the window moves at each step (larger = faster, more downsampling).
MaxPooling2D	pool_ size	(2,2)	Size of the pooling window

	strides	(2,2)	How much the window shifts; usually matches pool_size for downsampling by 2
GlobalMaxPooling2D	–	–	Collapses (spatially pools) each channel to one value by taking max over entire spatial dimension
Flatten	–	–	Reshapes multidimensional input into a 1D vector for Dense layers
Dense	kernel	(512×1024), (1024×4)	Fully-connected weights, shape depends on input/output units (e.g. input_dim × output_dim)
	bias	(1024), (4)	Bias added per output unit
	activation	relu, softmax	relu for hidden layers, softmax for final multi-class probabilities (classification tasks).
	units	1024, 4	Number of neurons/outputs in layer.
Dropout	rate	0.25	Fraction of inputs to set to zero during training (regularization, helps prevent overfitting)

Table A.2. CNN Approach 2 Model Parameters

Layer/Type	Parameter	Value	Description
Conv2D	kernel	(3×3×64), (3×3×128)	This filter is used to extract features from images.
	bias	(64), (128)	A per-unit offset that helps the neuron's output, allowing networks to fit the data more flexibly.
	activation	relu (Rectified Linear Unit)	Suppresses negative values, retaining positive ones (enabling deep networks to learn efficiently).
	dilation_ rate	(1,1)	Controls the spacing between kernel elements, (1,1) means no dilation (standard)
	filters	64, 128	Number of kernels (feature detectors) in a convolutional layer; determines how many features are extracted.
	groups	1	Number of groups for grouped convolution; 1 means standard convolution
	padding	Same	"same" keeps output size the same as input by adding zero-padding as needed
	kernel_size	(3,3)	Height and width of the convolution window.
	strides	(1,1)	How many pixels the window moves at each step (larger = faster, more downsampling).
MaxPooling2D	pool_size	(2,2)	Size of the pooling window
	strides	(2,2)	How much the window shifts; usually matches pool_size for downsampling by 2
GlobalMaxPooling2D	–	–	Collapses (spatially pools) each channel to one value by taking max over entire spatial dimension
Flatten	–	–	Reshapes multidimensional input into a 1D vector for Dense layers
Dense	kernel	(128×512), (512×4)	Fully-connected weights, shape depends on input/output units (e.g. input_dim × output_dim)
	bias	(512), (4)	Bias added per output unit

	activation	relu, softmax	relu for hidden layers, softmax for final multi-class probabilities (classification tasks).
	units	512, 4	Number of neurons/outputs in layer.
Dropout	rate	0.25	Fraction of inputs to set to zero during training (regularization, helps prevent overfitting)

This table presents the two LSTM experimental configurations explored before final model selection. These experiments were conducted to evaluate the effect of various architectural and training parameters on model performance. Only the optimal configuration was selected for use, while the other trials, which demonstrated inferior performance, are reported for documentation and reference purposes.

Table B.1. LSTM Approach 1 Model Parameters.

Layer/Type	Parameter	Value	Description
LSTM	kernel	(3×32)	The main weight matrix of the layer (maps inputs to outputs).
	recurrent_ kernel	(8×32)	Weights used for recurrent connections in LSTM (previous output to current).
	bias	(32)	Offsets added to the output after the weighted sum.
	recurrent_ activation	sigmoid	Nonlinear activation for recurrent gate operations (LSTM/GRU).
	units	8	Number of neurons or outputs in the layer.
	zero_ output_for_ma sk	False	If true, outputs zero values for masked inputs (sequences).
	activation	Tanh	Nonlinear function applied to outputs (e.g., relu, tanh, softmax).
batch_normaliz ation	gamma	(8)	Scale parameter vector for batch normalization
	beta	(8)	Shift parameter vector for batch normalization
	moving_ mean	(8)	Running mean for normalization
	moving_ variance	(8)	Running variance for normalization
	synchronized	False	If true, synchronizes mean/variance statistics across replicas
dropout	rate	0.3	Fraction of the input units to drop (randomly set to zero)
dense	kernel	(8×16)	Weight matrix (input dim × output units)
	bias	(16)	Bias vector (length = output units)
	activation	Relu	Activation function applied to outputs
	units	16	Number of neurons (output units)
dense_1	kernel	(16×32)	Weight matrix
	bias	(32)	Bias vector
	activation	Relu	Activation function
	units	32	Number of neurons
dropout_1	rate	0.3	Dropout rate (see above)
dense_2	kernel	(32×10)	Weight matrix
	bias	(10)	Bias vector
	activation	softmax	Softmax activation (for probabilities over 10 classes)

	units	10	Number of output classes/neurons (final output size)

Table B.2. LSTM Approach 2 Model Parameters.

Layer/Type	Parameter	Value	Description
LSTM	kernel	(3×16), (3×32)	The main weight matrix of the layer (maps inputs to outputs).
	recurrent_ kernel	(8×32)	Weights used for recurrent connections in LSTM (previous output to current).
	bias	(32)	Offsets added to the output after the weighted sum.
	recurrent_ activation	sigmoid	Nonlinear activation for recurrent gate operations (LSTM/GRU).
	units	8	Number of neurons or outputs in the layer.
	zero_ output_for_ma sk	False	If true, outputs zero values for masked inputs (sequences).
	activation	Tanh	Nonlinear function applied to outputs (e.g., relu, tanh, softmax).
batch_normaliz ation	gamma	(8)	Scale parameter vector for batch normalization
	beta	(8)	Shift parameter vector for batch normalization
	moving_ mean	(8)	Running mean for normalization
	moving_ variance	(8)	Running variance for normalization
	synchronized	False	If true, synchronizes mean/variance statistics across replicas
dropout	rate	0.3	Fraction of the input units to drop (randomly set to zero)
dense	kernel	(8×16)	Weight matrix (input dim × output units)
	bias	(16)	Bias vector (length = output units)
	activation	Relu	Activation function applied to outputs
	units	16	Number of neurons (output units)
dense_1	kernel	(16×32)	Weight matrix
	bias	(32)	Bias vector
	activation	Relu	Activation function
	units	32	Number of neurons
dropout_1	rate	0.3	Dropout rate (see above)
dense_2	kernel	(32×10)	Weight matrix
	bias	(10)	Bias vector
	activation	softmax	Softmax activation (for probabilities over 10 classes)
	units	10	Number of output classes/neurons (final output size)

This table presents one of the hybrid model experimental configurations explored before final model selection. These experiments were conducted to evaluate the effect of various architectural and training parameters on model performance. Only the optimal configuration was selected for use , while the other trials, which demonstrated inferior performance, are reported for documentation and reference purposes.

Table C.1. CNN, LSTM and GRU Approach 1 Model Parameters

Layer/Type	Parameter	Value	Description
Conv2D (block conv)	kernel	(3×3×64), (3×3×128), (3×3×256), (3×3×512).	Main weight matrix mapping inputs to outputs.
	bias	(64), (128), (256), (512).	Offsets added to layer outputs; helps model learn shifting the function.
	activation	relu	Non-linear function applied to output of the convolution
MaxPooling2D	strides	2×2	Step size for moving the pooling window
	pool size	2×2	Control area and step size for pooling downsampling.
GlobalAvgPooling2D	_	_	Collapses spatial dims by averaging across height & width
Reshape	target shape	(1, 512)	Reshapes tensor to desired output shape
LSTM	kernel	(512×1024), (256×512), (128×512).	Weights for input-to-hidden transformation
	recurrent_ kernel	(256×1024), (128×512).	Weights for hidden-to-hidden (recurrent) transformation
	bias	(1024), (512).	Bias vector for LSTM
	activation	tanh	Output activation function for LSTM
	recurrent_ activation	sigmoid	Activation for gates in LSTM
	units	256, 128.	Number of memory cells/units in the LSTM layer
	zero_ output_for_mask	false	Handles sequence masking for variable-length data
	return_ sequences	true	Set to return the full sequence (not just last output)
GRU	kernel	(128×384).	Input-to-hidden weight matrix
	recurrent_ kernel	(128×384).	Hidden-to-hidden weight matrix
	bias	(2×384).	Bias vector for the GRU
	activation	tanh	Output activation function for GRU
	recurrent_ activation	sigmoid	Activation for gates in GRU
	units	128.	Number of memory cells/units in the GRU layer
	zero_ output_for_mask	false	Handles sequence masking for variable-length data
BatchNormalization	gamma	(128)	Scaling parameter for normalized features (learnable)
	beta	(128)	Shifting parameter for normalized features (learnable)
	moving_ mean	(128)	Running mean (estimated during training, used at inference)
	moving_variance	(128)	Running variance (estimated during training, used at inference)
	synchronized	false	If using distributed training, stats are synchronized

Dropout	rate	0.2	Fraction of input units to set to zero during training (regularization)
Dense	kernel	(128×128), (128×10)	Weight matrix mapping input to output
	bias	(128), (10)	Bias vector added to output
	activation	relu, softmax	Nonlinear function applied to output; softmax for final classification